
UNIT 14 MOBILE INTERNET APPLICATIONS

- 14.0 Introduction
 - 14.0.1 Mobile Application Development\
 - 14.0.2 Components Of Enterprise Application
- 14.1 Objectives
- 14.2 Introduction To Xml
 - 14.2.1 Xml Database
 - 14.2.2 Xml Parsing
- 14.3 Handheld Device Markup
 - 14.3.1 Hand-Held Device Markup Language (Hdml)
 - 14.3.2 Wireless Markup Language (Wml)
- 14.4 Hypertext Markup Language
 - 14.4.1 Html5
- 14.5 Summary
- 14.6 Solutions/Answers
- 14.7 Further Readings

14.0 INTRODUCTION

Markup language will generate page layout , presentation components and enable us to interact with the server. Mobile app is a software that runs on the mobile device that provides the user interface for interaction.

For web pages the most popular markup language is HTML (hypertext markup language). HTML defines the web page components so that the browsers can render the web page as per the specification. HTML also posts the client requests to the server and gets them server response.

Mobile applications use XML for managing the data. Mobile devices use that networks like 3G 4G 5G and protocols like wireless application protocol (WAP) to connect to the internet. The connecting devices use xml-based wireless markup language (WML) for exchanging information.

14.0.1 Mobile Application development

Mobile application provides the mobile user with a graphical user interface and intuitive interfaces to get the required information. Modern mobile applications and use n tier architecture where the mobile app acts as a client and the server is deployed on a remote machine the server managers are huge database and provides API is to expose the data to the mobile client.

There are primarily two to main development platforms for mobile app iOS and Android. Android mobile apps can be developed in Java and the iOS mobile apps are developed in frameworks such as swift UI and UI kit.

Mobile app provides the below given functionality:

1. Creation of a graphical user interface
2. Allowing the end user to use the gestures, clicks to interact with application
3. Interface with remote service to get the data on demand.

How to develop the mobile app we need to follow the below given steps:

1. Select the mobile platform and the integrated development environment (IDE) to develop the mobile app in the selected language.
2. Develop the screen designs user interfaces libraries and integrations for the mobile app
3. Iteratively test the mobile app to ensure that the app conforms to the specifications
4. Package the mobile application for the platform
5. Host and distribute the application to the app Marketplace
6. Deploy the application to the end mobile device
7. Monitor the mobile performance and other related metrics

14.0.2 Components of Enterprise application

Let us look at the structure of the enterprise applications in detail in this section. We need to understand various layers of a typical enterprise application and the responsibility of each of those layers

Figure 1 provides the layer-wise components in a typical enterprise application. The channels provide the users access to various modes for interacting with the enterprise application. The user experience layer in the mobile app provides various capabilities for end users to interact. The API layer exposes the business capabilities to the presentation layer. The integration layer interacts with the backend system to get the enterprise data.

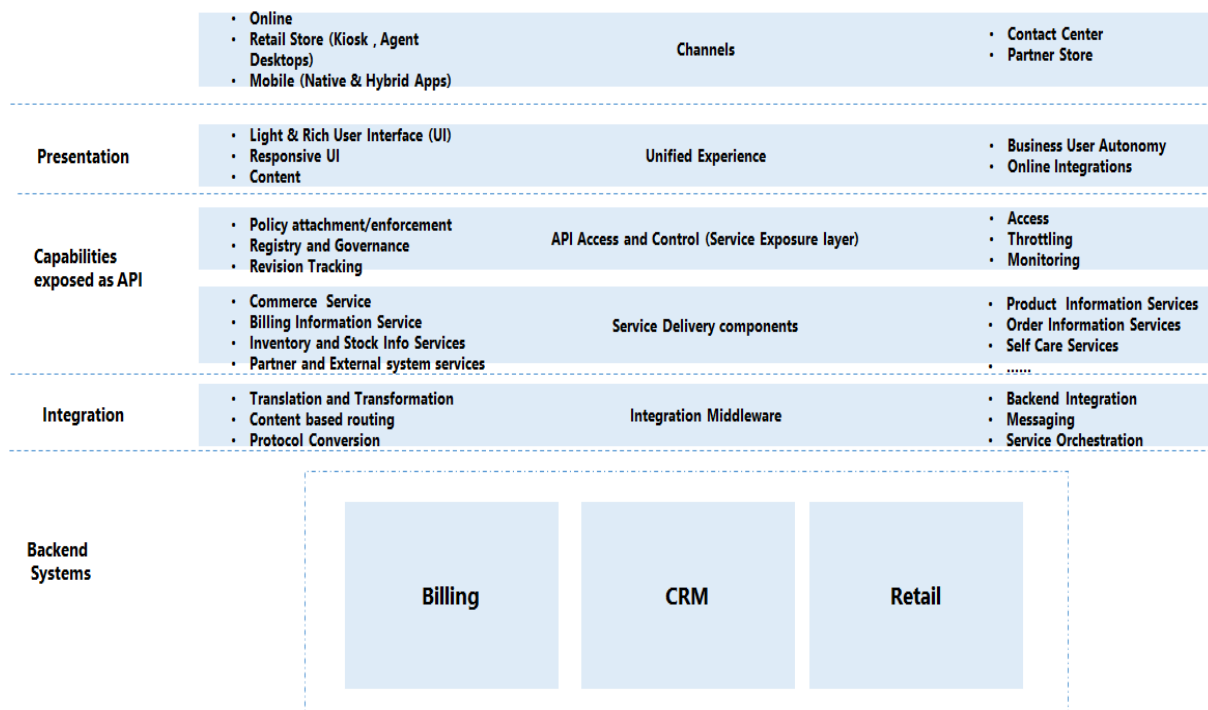


Figure 1 Layer wise Enterprise Application components

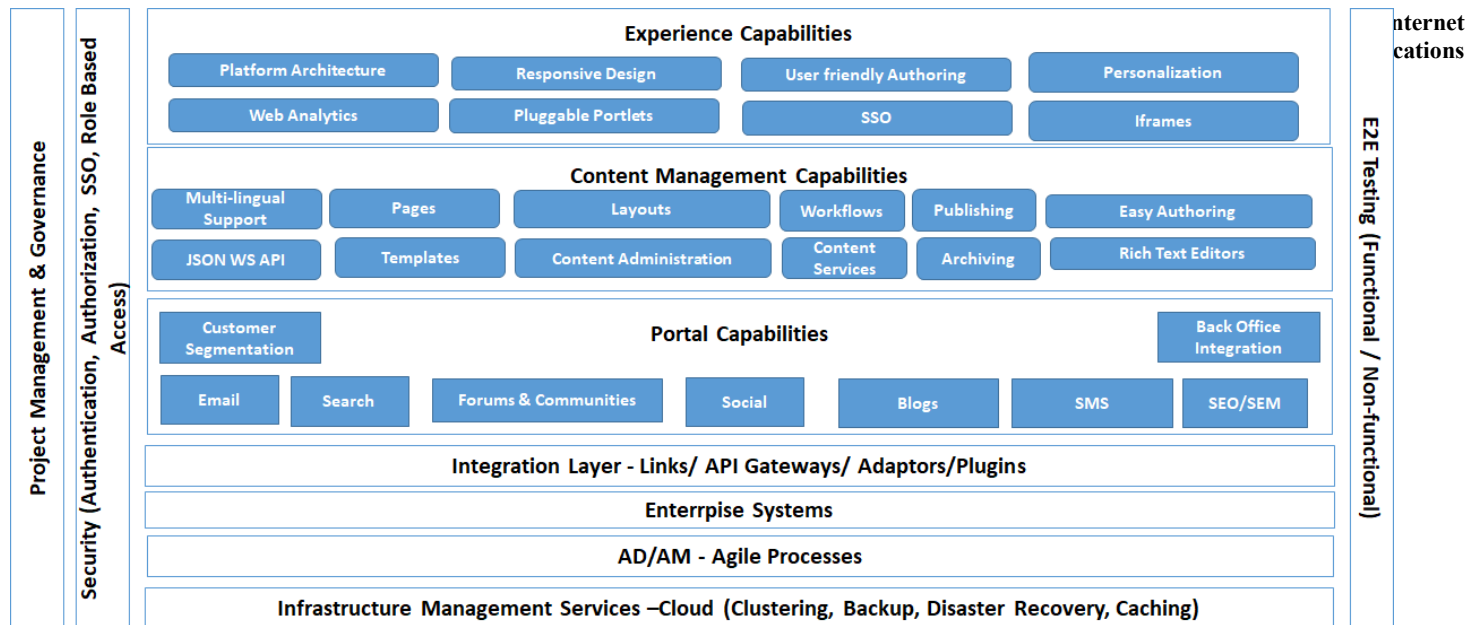


Figure 2 Content management application

In a content driven application the server layer mainly consists of content management features such as layouts, workflows, authoring etc. as depicted in Figure 2. Personalization, web analytics and responsive design are main features in the presentation layer.

The detailed components for each layer for an insurance application is depicted in Figure 3.

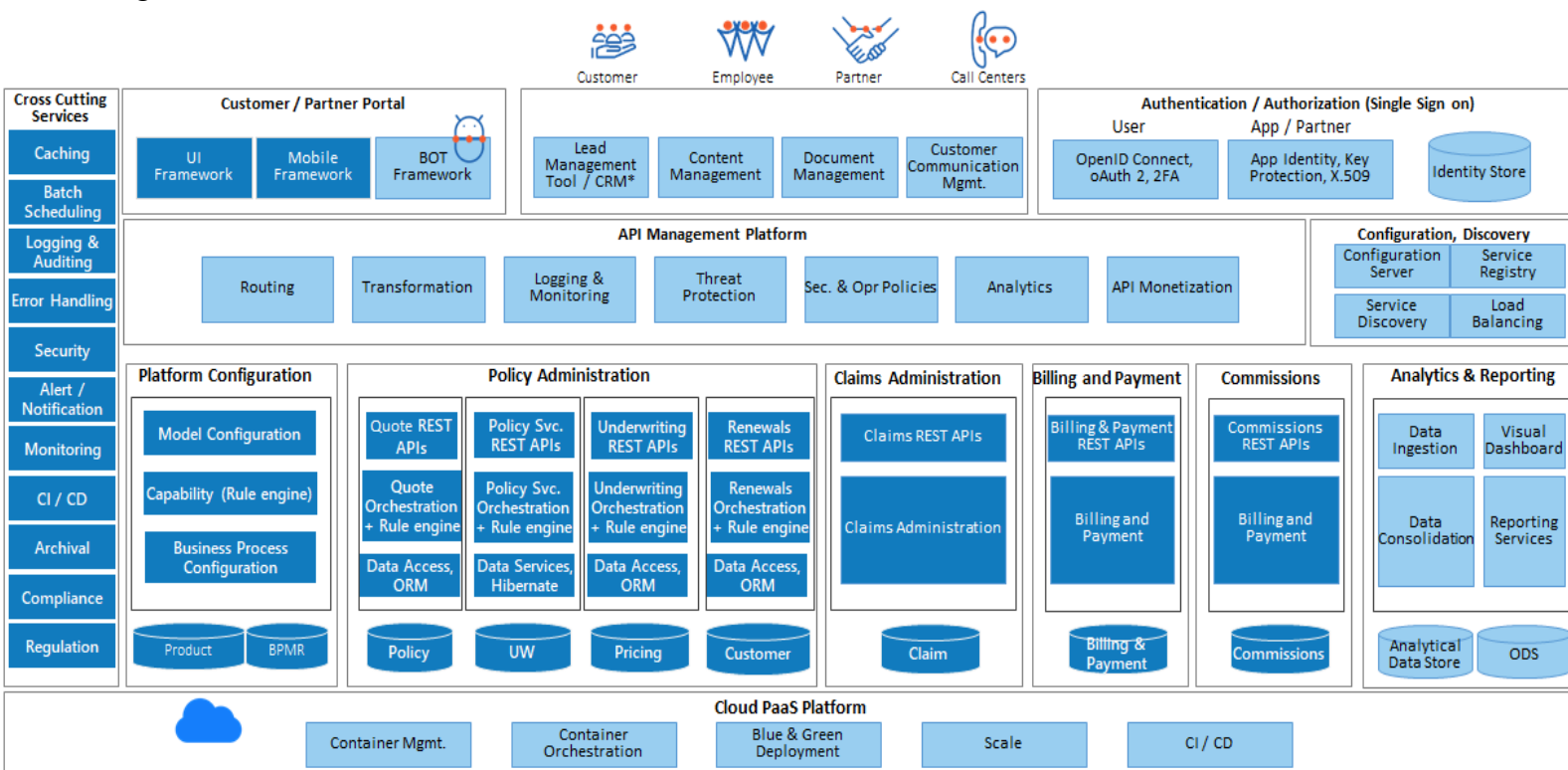


Figure 3 Layer wise components for Insurance application

The business components mainly consists of policy administration, claims management, billing and payment, commissions and analytic and reporting as depicted in Figure 3.

The main enterprise systems are depicted in Figure 4. In a typical enterprise application we have document management system, content management system, workflows, enterprise apps, collaboration apps and services. The middleware provides the services on top of the enterprise systems. The front end component such as login, dashboards, collaboration UI are part of the web page and the mobile app

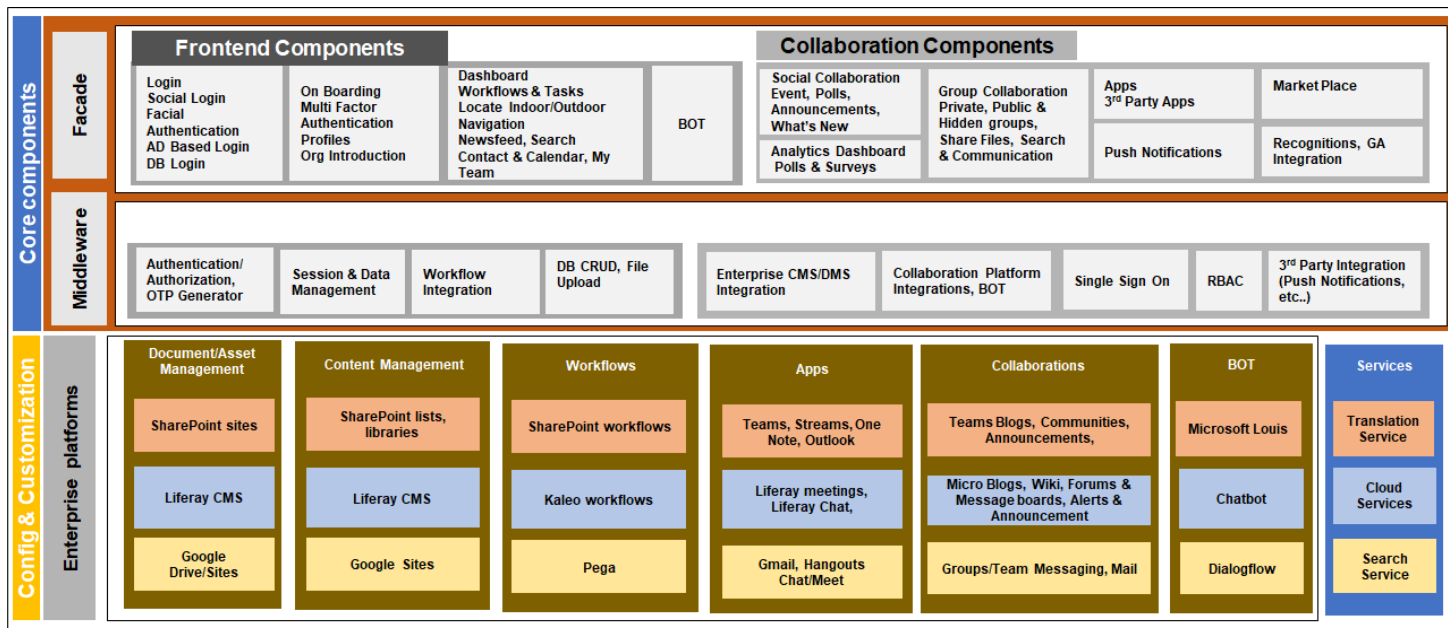


Figure 4 Enterprise Systems

14.1 OBJECTIVES

After going through this unit, you should be able to

- understand key concepts of XML,
- understand various types of device markups,
- understand the details of WML,
- understand HTML5 standard

14.2 INTRODUCTION TO XML

Extensible markup language (XML) is a declarative markup language uses standard set of tag and attributes for the presenting the data. The tag defines The high-level element entity and attributes define the properties of the entity. There are multiple XML based standards that's popular for data exchange and data transmission. XML is a platform and language independent structure format.

XML format is used in variety of scenarios. Firstly XML helps us to manage the data in structured way using well-defined tags and attributes. We can also define the information rules and constraints. XML is also used to share the data across

two systems using standard formats. Servers can push the data to the mobile clients using XML data format. We can also represent the object relationship and hierarchies in XML format. XML can be used as a database where we manage the information and XML can also be used to specify the commands, define the page layout, specify the constraints, enforce the rules, establish the relationships between entities. As XML is extensible, we can also add custom tags, custom attributes and elements as part of the XML.

We have given a sample XML representing a list of books below.

```
<?xml version="1.0" encoding="UTF-8"?>
<books>
  <book>
    <name>Gandhi: An Autobiography</name>
    <author>Mahatma Gandhi</author>
    <language>English</language>
    <genre>Autobiography</genre>
  </book>
  <book>
    <name>Letters from a Father to His Daughter</name>
    <author>Jawaharlal Nehru</author>
    <language>English</language>
    <genre>Non Fiction</genre>
  </book>
</books>
```

The XML represents a list of books where book is the main entity. The book entity is represented by a <book> tag. Various properties of the book entity such as name order and General or a presented by various attributes within the tag.

We can also define the schema for an XML. We can define various elements and the constraints for the elements using XML Schema Definition (XSD). We use XSLT (Extensible Stylesheet Language Transformations) to transform one XML document into another. We can use data type definition (DTD) for XML file validation. DTD can be used to specify the rules for the XML document; we can specify rules such as root element, nesting structure and others using the DTD.

14.2.1 XML Database

We can define the data in XML database. Given below is the representation of the students database in the table format we have to find various attributes of student such as name course taken phone number and address. Once we define the data in XML database we can use the XML parsers to read the data and query the data

using the filters and business logic. We can parse XML data into a list of key value pairs.

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student>
    <name>Kumar</name>
    <course>MBA</course>
    <phone>1231231234</phone>
    <address>New Delhi</address>
  </student>
  <student>
    <name>Michael</name>
    <course>MCA</course>
    <phone>1331241234</phone>
    <address>Bengaluru</address>
  </student>
  <student>
    <name>Amar</name>
    <course>MA</course>
    <phone>1331231434</phone>
    <address>Lucknow</address>
  </student>
</students>
```

Many systems store the data in the XML database. As XML format is structured and well-defined, it is also used to exchange the data across various systems.

14.2.2 XML Parsing

Parsing an XML document creates hashtable of key value pairs. The parser understands the encoding used in an XML document, validates the XML documents, and then passes all the tags and attributes in the XML document. The parser gets the value mentioned between the tags and between the attributes. The parser uses DTD for XML validation. State various rules such as the constraint validation, the root element validation, and encoding using the DTD.

There are mainly two kinds of parsers – SAX parser and DOM parser. The SAX parser is an event-driven parser that parses the XML data sequentially. The XML tags are event sources. The SAX parser is faster as compared to the DOM parser as the SAX parser need not parse the entire document.

The DOM parser uses the document object model (DOM) which is a tree-structure representing the XML data. DOM parser parses the entire XML document as a DOM tree and stores the XML information hierarchically. DOM format supports xpath that can be used to query the hierarchical information.

Let us look at an example of SAX parser for a sample XML file

```
<students>
  <student regno="1">
    <firstname>Shiva</firstname>
    <lastname>Kumar</lastname>
  </student>
  <student regno="2">
    <firstname>Ram</firstname>
    <lastname>Krishna</lastname>
  </student>
</students>
```

The above XML file lists the details of students. To use the SAX parser we create a model class for Student entity. The model class is essentially a POJO (Plain Old Java Object) that stores the value of the student entity. In this case, we store three attributes of the student – regno, firstname and lastname.

We have given the sample SAX parser code to parse the students data in the above XML.

```
package com.example;

import java.util.ArrayList;
import java.util.Stack;

import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;

public class StudentParserHandler extends DefaultHandler
{
    //This is the list which shall be populated while parsing the XML.
    private ArrayList studentList = new ArrayList();

    //As we read any XML element we will push that in this stack
    private Stack elementStack = new Stack();

    //As we complete one user block in XML, we will push the User instance in
    userList
```

```
private Stack objectStack = new Stack();

public void startDocument() throws SAXException
{

}

public void endDocument() throws SAXException
{

}

public void startElement(String uri, String localName, String qName,
    Attributes attributes) throws SAXException
{
    //Push it in element stack
    this.elementStack.push(qName);

    //If this is start of 'student' element then prepare a new Student instance and
    push it in object stack
    if ("student".equals(qName))
    {
        //New User instance
        Student s= new Student();

        //Set all required attributes in any XML element here itself
        if(attributes != null && attributes.getLength() == 1)
        {
            s.setRollno(Integer.parseInt(attributes.getValue(0)));
        }
        this.objectStack.push(s);
    }
}

public void endElement(String uri, String localName, String qName) throws
    SAXException
{
    //Remove last added element
```



```

this.elementStack.pop();

//User instance has been constructed so pop it from object stack and push in
userList
if ("student".equals(qName))
{
    Student o = this.objectStack.pop();
    this.studentList.add(o);
}
}

/**
 * This will be called everytime parser encounter a value node
 * */
public void characters(char[] ch, int start, int length) throws SAXException
{
    String value = new String(ch, start, length).trim();

    if (value.length() == 0)
    {
        return; // ignore white space
    }

    //handle the value based on to which element it belongs
    if ("firstName".equals(currentElement()))
    {
        Student s = (Student) this.objectStack.peek();
        s.setFirstName(value);
    }
    else if ("lastName".equals(currentElement()))
    {
        Student s = (Student) this.objectStack.peek();
        s.setLastName(value);
    }
}

/**
 * Utility method for getting the current element in processing

```

```
    */  
    private String currentElement()  
    {  
        return this.elementStack.peek();  
    }  
  
    //Accessor for userList object  
    public ArrayList getUsers()  
    {  
        return studentList;  
    }  
}
```

Multiple languages such as WML, SyncML, VoiceXML etc. are XML-based languages. Mobile devices use these XML based languages for exchanging the data.

☛ Check Your Progress 1

1. _____ can be used to transform one XML document into another.
2. The XML rules are specified in _____
3. _____ parser is an event-driven parser
4. _____ parser uses tree-structure representing the XML data

14.3 HANDHELD DEVICE MARKUP

In this section we discuss about the HDML and WML markup languages.

14.3.1 Hand-held Device Markup Language (HDML)

Handheld device markup language is type of markup language invented for small wireless and handheld devices. The HDML is mainly used in the handheld devices like PDA, mobile phones and others.

The start tag is HDML that indicates the start of the deck. Because you specify the actions within HDML tag. Within each deck there are a list of cards that can also specify the actions. There are various kinds of card elements such as display card element, nodisplay card element and such. The detailed list of specifications can be found at <https://www.w3.org/TR/NOTE-Submission-HDML-spec.html>

HDML does not support scripts.

14.3.2 Wireless Markup Language (WML)

WML is XML-based W3C standard for wireless devices. Similar to XML, WML has DTD and XML-based syntax for specifying elements. WML supports script where we can code loops, procedures and conditions.

The WAP browser that runs on a mobile device renders the WML cards. Similar to a HTML page, each WML card has links, text, list and other elements as given below:

- Images
- Tables
- Anchor elements
- Formatted text
- Scripts
- Events

A sample WML deck with two cards are given below -

```
<?xml version="1.0"?>
<!DOCTYPE wml PUBLIC "-//WAPFORUM//DTD WML 1.2//EN"
"http://www.wapforum.org/DTD/wml12.dtd">
<wml>
  <card id="home" title="Home Card">
    <p>
Welcome to Home card

    </p>
  </card>
  <card id="product" title="Product Card">
    <p>
Welcome to the product card

    </p>
  </card>
</wml>
```

The WML deck has main cards as shown above. Each card has an id, title and the content attributes.

14.4 HYPERTEXT MARKUP LANGUAGE

Hypertext markup language (HTML) is the most popular standard for rendering internet web pages. We cannot there that a web page with all the text images and hyperlinks using them HTML standard. The term hypertext refers to jumping from one page to another through hyperlink.

We can format the HTML webpage using the style sheets. Various elements that are as input textbox list links images buttons anchors tables 2 blocks frames forms can be rendered using the HTML webpage. You can use JavaScript to inject dynamic behavior into the webpage such as event handling request-response management animation input validation form submission and so on.

Cascading style sheets (CSS) define the style specification for HTML element. For instance, the stylesheet specifies the width height alignment for an HTML element.

The most popular HTML element is the form and input type. We can use JavaScript to validate the form elements for us type validation length validation and so on.

There are primarily two types of HTML pages static web page is the one where the page content remain static until the page is edited. A dynamic web page changes the content based on attribute such as the user input, user profile and other details. We use the JavaScript to get the dynamic content for the dynamic web page.

We have depicted the sample HTML code below. All the HTML elements are declared within <body> element. The page heading is declared within <h1> tag and the image is specified with tag. The table is declared using <table> tag.

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Page Heading</h1>
    <p>Sample Paragraph</p>
    
    <table>
      <tr>
        <td>Row Data</td>
      </tr>
    </table>
    <h2>Sample HTML list</h2>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
    </ul>
    <div style="border: 1px solid black">Hello World</div>
  </body>
</html>
```

14.4.1 HTML5

HTML5 is the latest version of the HTML standard. It is enhancement over html4 that provides various instruments for multimedia edition real-time streaming and web sockets. HTML5 provides interoperable specifications for current Browsers and allows error handling features. HTML5 adds various elements for enhanced page performance and User experience and it provides enhanced forms and form controls.

HTML5 also provides audio and video HTML elements with controls for playing seeking, pausing, aborting and others. We cab also do real time streaming using HTML5 standard. HTML5 provides enhanced form and input types. The Canvas element provides the ability for 2D drawing. HTML5 also provides scalable

vector Graphics for drawing. HTML5 also provides APIs for sending and receiving messages

☛ Check Your Progress 2

1. True or False: HDML support scripts
2. HTML webpage can be formatted using the _____
3. In HTML, the page heading is specified using _____ tag
4. Dynamic content is retrieved through _____
5. Image is specified using _____ tag

14.5 SUMMARY

In this unit, we started discussing the main components of an enterprise application. We looked at various layers and the layer-wise components of enterprise application. We then discussed the XML format and the applications of the XML standard. We discussed the SAX and DOM parser for parsing the XML data. We then discussed the HDML and WML standards that is used in the mobile devices. Finally we discussed various HTML elements and the enhancements of HTML5 standard.

14.6 SOLUTIONS/ANSWERS

Check Your Progress 1

1. XSLT
2. DTD
3. SAX parser
4. DOM Parser

Check Your Progress 2

1. False
2. Cascading style sheets (CSS)
3. <h1>
4. JavaScript
5.

14.7 FURTHER READINGS

References

Mobile Computing 3rd Edition by Raj Kamal -

<https://www.amazon.in/Mobile-Computing-Raj-Kamal/dp/0199455414>

<https://www.w3.org/TR/NOTE-Submission-HDML-spec.html>